

TUGAS BESAR ARTIFICIAL NEURAL NETWORK (ANN)

Disusun untuk memenuhi Ujian Akhir Semester Mata Kuliah Artificial Intelligent

Dosen Pengampu: Dr.-Ing. Ahmad Z. Ihsan



Oleh:

Davina Shakira

2303030029

PROGRAM STUDI SISTEM TEKNOLOGI DAN INFORMASI

FAKULTAS SAINS DAN TEKNOLOGI UNIVERSITAS

DARUNNAJAH

2025-2026

Daftar Isi :

A. Deskripsi Persoalan	3
B. Pembahasan	5
i. Penjelasan Implementasi	5
ii. Deskripsi Kelas beserta deskripsi atribut dan methodnya	6
iii. Penjelasan Forward Propagation	9
iv. Penjelasan Backward Propagation dan Weight Update	11
B. Hasil Pengujian.....	13
i. Pengaruh Depth Dan Width	13
ii. Pengaruh Fungsi Aktivasi	14
iii. Pengaruh Learning Rate	14
iv. Pengaruh Inisialisasi Bobot	15
v. Pengaruh Regularisasi.....	15
vi. Fungsi Loss	15
vii. Pengaruh Ukuran Batch (Batch Size).....	16
viii. Pengaruh Jumlah Epoch	16
ix. Metode Optimisasi	17
D. Perbandingan Dengan Library Keras	17
E. Kesimpulan Dan Saran	20
F. Referensi	22
G. LAMPIRAN	23

A. Deskripsi Persoalan

Pada tugas ini digunakan dataset Dry Bean sebagai objek klasifikasi untuk membangun model Artificial Neural Network (ANN) from scratch. Dry Bean Dataset merupakan dataset klasifikasi multiclass yang berisi karakteristik fisik dari beberapa jenis biji kacang kering (dry beans).

Dataset ini bertujuan untuk mengelompokkan biji kacang ke dalam kelas yang sesuai berdasarkan atribut bentuk dan ukuran yang dimiliki. Setiap data merepresentasikan satu objek biji kacang dan terdiri dari sejumlah atribut numerik yang menggambarkan karakteristik dari objek tersebut.

Dataset Dry Bean memiliki total 13.611 data dan terdiri dari 16 atribut numerik sebagai fitur masukan serta 1 atribut target sebagai label kelas.

Atribut pada dataset meliputi:

1. Area → luas objek biji dalam piksel
2. Perimeter → panjang keliling objek
3. MajorAxisLength → panjang sumbu utama
4. MinorAxisLength → panjang sumbu minor
5. AspectRatio → rasio panjang dan lebar
6. Eccentricity → tingkat penyimpangan bentuk dari lingkaran
7. ConvexArea → luas convex hull
8. EquivDiameter → diameter ekuivalen
9. Extent → rasio area terhadap bounding box
10. Solidity → rasio area terhadap convex area
11. Roundness → tingkat kebulatan objek
12. Compactness → tingkat kepadatan bentuk
13. ShapeFactor1
14. ShapeFactor2
15. ShapeFactor3
16. ShapeFactor4

Label target terdiri dari 7 kelas jenis kacang:

1. Seker
2. Barbunya
3. Bombay
4. Cali
5. Dermason
6. Horoz
7. Sira

Permasalahan pada penelitian ini adalah bagaimana membangun model Artificial Neural Network dari nol (from scratch) yang mampu mengenali pola antar atribut dan melakukan klasifikasi jenis biji kacang secara akurat tanpa menggunakan library machine learning siap pakai.

Tahap awal dilakukan preprocessing data yang meliputi pemisahan fitur dan label, transformasi label menjadi bentuk numerik menggunakan encoding, standarisasi data, serta pembagian data menjadi data training dan data testing.

Setelah data dipersiapkan, dibangun model Artificial Neural Network menggunakan pendekatan matematis dasar jaringan saraf. Setiap neuron menerima masukan dari atribut dataset kemudian menghitung transformasi linear:

$$Z = x_1W_1 + x_2W_2 + \dots + x_nW_n + b$$

atau dalam bentuk matriks:

$$Z = XW + b$$

Keterangan:

- X = matriks fitur input
- W = bobot
- b = bias
- Z = hasil transformasi linear

Hasil transformasi kemudian diproses menggunakan fungsi aktivasi:

$$A = f(Z)$$

Fungsi aktivasi bertujuan memberikan kemampuan model untuk mempelajari hubungan non-linear antar atribut.

Pada output layer digunakan fungsi Softmax untuk menghasilkan probabilitas setiap kelas:

$$\hat{y}_i = \exp(z_i) / \sum \exp(z_j)$$

Karena kasus yang diselesaikan merupakan klasifikasi multiclass dengan satu label benar pada setiap data, digunakan fungsi loss Multiclass Cross Entropy:

$$L = - \sum y \log(\hat{y})$$

Nilai loss digunakan untuk menghitung gradien pada proses backward propagation.

Gradien bobot dihitung menggunakan:

$$dW = \partial L / \partial W$$

Kemudian parameter diperbarui menggunakan algoritma Gradient Descent:

$$W_{\text{new}} = W - \alpha(dW)$$

$$b_{\text{new}} = b - \alpha(db)$$

dengan α sebagai learning rate.

Proses training dilakukan secara berulang selama sejumlah epoch hingga model memperoleh kemampuan klasifikasi yang optimal. Untuk memperoleh konfigurasi terbaik dilakukan eksperimen terhadap berbagai parameter seperti jumlah hidden layer, jumlah neuron, inisialisasi bobot, fungsi aktivasi, fungsi loss, learning rate, metode optimisasi, ukuran batch, dan jumlah epoch.

B. Pembahasan

i. Penjelasan Implementasi

Pada penelitian ini dilakukan implementasi Artificial Neural Network (ANN) secara from scratch menggunakan bahasa pemrograman Python dan library NumPy sebagai alat bantu operasi numerik. Implementasi dilakukan tanpa menggunakan library machine learning siap pakai seperti Scikit-Learn maupun TensorFlow pada proses pembelajaran model sehingga seluruh proses komputasi jaringan saraf dibangun secara manual.

Tahapan implementasi dimulai dari proses persiapan dataset Dry Bean. Dataset dibaca kemudian dipisahkan menjadi atribut (fitur) dan label (target). Fitur yang digunakan terdiri dari 16 atribut numerik yang merepresentasikan karakteristik geometris dari biji kacang, sedangkan target berupa 7 kelas jenis kacang.

Setelah pemisahan data dilakukan, label kelas diubah menjadi representasi numerik menggunakan proses encoding agar dapat diproses oleh model ANN. Selanjutnya dataset dibagi menjadi data training dan data testing untuk keperluan pelatihan dan evaluasi model.

Model Artificial Neural Network yang dibangun terdiri dari tiga komponen utama yaitu input layer, hidden layer, dan output layer.

Input layer menerima data fitur dari dataset dengan jumlah neuron mengikuti jumlah atribut yang digunakan yaitu sebanyak 16 neuron.

$$\text{Input} = [x_1, x_2, x_3, \dots, x_{16}]$$

Data kemudian diteruskan menuju hidden layer yang bertugas melakukan ekstraksi pola dari data. Pada penelitian ini dilakukan eksperimen jumlah hidden layer dan jumlah neuron untuk memperoleh arsitektur terbaik.

Setiap neuron melakukan transformasi linear menggunakan persamaan:

$$Z = XW + b$$

Keterangan:

- X = data input
- W = matriks bobot
- b = bias
- Z = hasil transformasi linear

Output transformasi linear kemudian diproses menggunakan fungsi aktivasi untuk menghasilkan hubungan non-linear.

$$A = f(Z)$$

Fungsi aktivasi yang diuji pada penelitian ini meliputi:

1. Sigmoid
2. ReLU

3. Tanh

Setelah data melewati hidden layer, hasil aktivasi diteruskan menuju output layer.

Karena penelitian ini menyelesaikan kasus klasifikasi multiclass dengan 7 kelas, maka digunakan 7 neuron output.

$$\text{Output Layer} = [y_1, y_2, y_3, y_4, y_5, y_6, y_7]$$

Pada output layer digunakan fungsi Softmax untuk mengubah nilai keluaran menjadi probabilitas:

$$\hat{y}_i = \exp(z_i) / \sum \exp(z_j)$$

Kelas dengan probabilitas terbesar dipilih sebagai hasil prediksi model.

Untuk mengukur kesalahan prediksi digunakan fungsi Multiclass Cross Entropy:

$$L = - \sum y \log(\hat{y})$$

Nilai loss yang dihasilkan kemudian digunakan pada proses backward propagation untuk menghitung perubahan bobot.

Gradien dihitung menggunakan:

$$dW = \partial L / \partial W$$

$$db = \partial L / \partial b$$

Parameter diperbarui menggunakan metode Gradient Descent:

$$W_{\text{new}} = W - \alpha(dW)$$

$$b_{\text{new}} = b - \alpha(db)$$

dengan:

- α = learning rate

Proses training dilakukan secara iteratif selama sejumlah epoch hingga model mencapai performa yang optimal.

Selain implementasi dasar, dilakukan eksperimen terhadap berbagai hyperparameter meliputi jumlah hidden layer, jumlah neuron, inisialisasi bobot, fungsi aktivasi, fungsi loss, learning rate, metode optimisasi, ukuran batch, dan jumlah epoch untuk memperoleh konfigurasi model terbaik.

ii. Deskripsi Kelas beserta deskripsi atribut dan methodnya

Pada implementasi Artificial Neural Network (ANN) from scratch, struktur program dibangun menggunakan kumpulan variabel dan fungsi yang saling terhubung untuk menjalankan proses pembelajaran model. Setiap komponen memiliki peran tertentu mulai dari pengolahan data hingga proses prediksi.

a. Atribut Program

1. X_train

Variabel yang digunakan untuk menyimpan data fitur training. Data ini digunakan selama proses pelatihan model untuk mempelajari pola klasifikasi.

Bentuk data:

$$X_{\text{train}} \in \mathbb{R}^{n \times 16}$$

Keterangan:

- n = jumlah data training
- 16 = jumlah atribut pada Dry Bean Dataset

2. X_{test}

Variabel yang menyimpan data fitur testing yang digunakan untuk mengevaluasi performa model setelah proses training selesai.

3. Y_{train}

Variabel yang menyimpan label target pada data training dalam bentuk numerik atau hasil encoding.

4. Y_{test}

Variabel yang menyimpan label target pada data testing untuk proses evaluasi model.

5. W (Weight)

Parameter utama jaringan saraf yang menentukan hubungan antar neuron.

Bobot direpresentasikan dalam bentuk matriks:

$$W \in \mathbb{R}^{i \times j}$$

Setiap iterasi training akan memperbarui nilai bobot agar error semakin kecil.

6. b (Bias)

Parameter tambahan yang digunakan untuk menyesuaikan hasil transformasi linear.

Persamaan neuron:

$$Z = XW + b$$

7. learning_rate

Parameter yang mengatur besar perubahan bobot pada setiap proses update.

Persamaan:

$$W_{\text{new}} = W - \alpha(dW)$$

dengan:

α = learning rate

8. epoch

Jumlah iterasi pelatihan terhadap seluruh dataset.

9. batch_size

Jumlah data yang diproses dalam satu kali update parameter pada metode Mini Batch Gradient Descent.

B. Method Program

1. $\text{forward_fcl}()$

Fungsi untuk menghitung proses forward propagation pada fully connected layer.

Persamaan:

$$Z = XW + b$$

Input:

- Data fitur

- Bobot
- Bias

Output:

- Nilai transformasi linear

2. sigmoid()

Fungsi aktivasi Sigmoid.

Persamaan:

$$\sigma(x) = 1/(1+e^{-x})$$

Rentang output:

$$0 \leq \sigma(x) \leq 1$$

3. relu()

Fungsi aktivasi ReLU.

Persamaan:

$$\text{ReLU}(x) = \max(0, x)$$

Digunakan untuk meningkatkan kemampuan model mempelajari hubungan non-linear.

4. tanh()

Fungsi aktivasi Hyperbolic Tangent.

Persamaan:

$$\tanh(x) = ((e^x - e^{-x}) / (e^x + e^{-x}))$$

Rentang output:

$$-1 \leq \tanh(x) \leq 1$$

5. softmax()

Digunakan pada output layer untuk menghasilkan probabilitas setiap kelas.

Persamaan:

$$\hat{y}_i = \exp(z_i) / \sum \exp(z_j)$$

6. multiclass_cross_entropy()

Digunakan untuk menghitung error pada klasifikasi multiclass.

Persamaan:

$$L = -\sum y \log(\hat{y})$$

7. backpropagation()

Method untuk menghitung gradien error terhadap parameter jaringan.

Output:

- $dW \rightarrow$ gradien bobot
- $db \rightarrow$ gradien bias

8. update_parameter()

Digunakan untuk memperbarui bobot dan bias berdasarkan hasil gradien.

Persamaan:

$$\begin{aligned} W_{\text{new}} &= W - \alpha(dW) \\ b_{\text{new}} &= b - \alpha(db) \end{aligned}$$

9. predict()

Digunakan untuk menghasilkan kelas prediksi berdasarkan probabilitas tertinggi dari output layer.

Persamaan:

$$\text{Prediction} = \text{argmax}(\hat{y})$$

iii. Penjelasan Forward Propagation

Forward Propagation merupakan proses utama dalam Artificial Neural Network (ANN) yang bertujuan untuk mengalirkan informasi dari input layer menuju output layer untuk menghasilkan prediksi kelas. Pada tahap ini data diproses secara berurutan melalui operasi matematika berupa transformasi linear dan fungsi aktivasi.

Pada penelitian ini digunakan dataset Dry Bean yang memiliki 16 atribut sehingga setiap data direpresentasikan sebagai vektor masukan:

$$X = [x_1, x_2, x_3, \dots, x_{16}]$$

Setiap fitur masukan kemudian dikalikan dengan bobot masing-masing neuron dan ditambahkan bias.

Secara matematis proses transformasi linear dituliskan sebagai:

$$Z = XW + b$$

Keterangan:

- X = matriks input
- W = matriks bobot
- b = bias
- Z = output transformasi linear

Jika dituliskan pada satu neuron:

$$Z = x_1w_1 + x_2w_2 + \dots + x_nw_n + b$$

Tahap ini bertujuan menghasilkan kombinasi numerik dari seluruh atribut yang akan digunakan sebagai dasar pengambilan keputusan pada jaringan.

Setelah transformasi linear selesai, hasilnya diteruskan menuju fungsi aktivasi.

$$A = f(Z)$$

Fungsi aktivasi digunakan agar jaringan saraf mampu mempelajari hubungan non-linear pada data. Tanpa fungsi aktivasi, seluruh jaringan hanya akan menghasilkan transformasi linear sehingga kemampuan klasifikasi menjadi terbatas.

Pada penelitian ini dilakukan pengujian tiga fungsi aktivasi.

1. Sigmoid

Persamaan:

$$\sigma(x) = 1/(1+e^{-x})$$

Karakteristik:

- Output antara 0–1

- Cocok untuk probabilitas
 - Rentan mengalami vanishing gradient
2. ReLU (Rectified Linear Unit)

Persamaan:

$$\text{ReLU}(x) = \max(0, x)$$

Karakteristik:

- Komputasi lebih sederhana
 - Mempercepat training
 - Mengurangi vanishing gradient
3. Tanh (Hyperbolic Tangent)

Persamaan:

$$\tanh(x) = ((e^x - e^{-x}) / (e^x + e^{-x}))$$

Karakteristik:

- Output antara -1 sampai 1
- Distribusi aktivasi lebih seimbang
- Memberikan performa terbaik pada penelitian ini

Setelah melewati hidden layer, hasil aktivasi diteruskan menuju output layer.

Karena penelitian ini merupakan klasifikasi multiclass dengan 7 kelas, maka digunakan fungsi Softmax.

Persamaan Softmax:

$$\hat{y}_i = \exp(z_i) / \sum \exp(z_j)$$

Keterangan:

- z_i = output neuron ke-i
- $\hat{y}_i$ = probabilitas kelas ke-i

Softmax mengubah seluruh output menjadi distribusi probabilitas dengan total nilai:

$$\sum \hat{y} = 1$$

Sebagai contoh:

Output sebelum Softmax:

[2.1, 0.8, 1.4]

Setelah Softmax:

[0.60, 0.16, 0.24]

Interpretasi:

- Kelas pertama memiliki probabilitas 60%
- Model memilih kelas pertama sebagai prediksi

Prediksi akhir diperoleh menggunakan fungsi:

$$\text{Prediction} = \text{argmax}(\hat{y})$$

yang memilih kelas dengan probabilitas terbesar.

Secara keseluruhan alur Forward Propagation dapat dituliskan sebagai:

Input Data --> Transformasi Linear ($Z = XW + b$) --> Fungsi Aktivasi ($A = f(Z)$) --> Output Layer --> Softmax --> Prediksi Kelas

Tahapan ini dilakukan berulang pada setiap iterasi training dan menjadi dasar sebelum dilakukan proses perhitungan error pada backward propagation.

iv. Penjelasan Backward Propagation dan Weight Update

Setelah proses Forward Propagation menghasilkan prediksi kelas, tahap berikutnya adalah Backward Propagation. Backward Propagation merupakan proses menghitung kesalahan prediksi kemudian menyebarkan error tersebut dari output layer menuju layer sebelumnya untuk memperbarui parameter model.

Tujuan utama dari proses ini adalah meminimalkan nilai error sehingga model dapat menghasilkan prediksi yang semakin akurat.

Tahap pertama dilakukan perhitungan fungsi loss.

Karena penelitian ini menggunakan klasifikasi multiclass, maka digunakan fungsi Multiclass Cross Entropy.

Persamaan fungsi loss:

$$L = - \sum y \log(\hat{y})$$

Keterangan:

- L = nilai loss
- y = label sebenarnya
- $\hat{y}$ = probabilitas hasil prediksi

Semakin kecil nilai loss maka semakin baik kemampuan model dalam melakukan prediksi.

Setelah loss diperoleh, dilakukan proses propagasi balik untuk menghitung pengaruh setiap parameter terhadap error.

Gradien pada output layer dihitung menggunakan:

$$\delta = \hat{y} - y$$

Keterangan:

- δ = error output
- $\hat{y}$ = probabilitas prediksi
- y = label sebenarnya

Nilai error tersebut digunakan untuk menghitung gradien bobot.

Gradien bobot:

$$dW = A^T \delta$$

Gradien bias:

$$db = \sum \delta$$

Keterangan:

- dW = perubahan bobot
- db = perubahan bias
- A = output aktivasi layer sebelumnya

Setelah gradien diperoleh dilakukan propagasi error menuju hidden layer.

Persamaan propagasi balik:

$$\delta_{\text{hidden}} = (\delta_{\text{output}} \times W^T) \odot f'(Z)$$

Keterangan:

- W^T = transpose bobot
- $f'(Z)$ = turunan fungsi aktivasi
- $\odot$ = perkalian elemen

Turunan fungsi aktivasi:

1. Sigmoid

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

2. ReLU

$$\text{ReLU}'(x) = 1, \text{ jika } x > 0$$

$$\text{ReLU}'(x) = 0, \text{ jika } x \leq 0$$

3. Tanh

$$\tanh'(x) = 1 - \tanh^2(x)$$

Setelah seluruh gradien diperoleh dilakukan pembaruan parameter menggunakan algoritma Gradient Descent.

Persamaan update bobot:

$$W_{\text{new}} = W - \alpha(dW)$$

Persamaan update bias:

$$b_{\text{new}} = b - \alpha(db)$$

Keterangan:

- α = learning rate
- dW = gradien bobot
- db = gradien bias

Learning rate menentukan besar langkah perubahan parameter.

Jika learning rate terlalu besar:

- training tidak stabil
- sulit konvergen

Jika terlalu kecil:

- training lambat

Pada penelitian ini dilakukan eksperimen learning rate dan diperoleh nilai terbaik sebesar:

$$\alpha = 0,1$$

Selain itu digunakan metode Mini Batch Gradient Descent sehingga pembaruan parameter dilakukan pada sebagian data dalam setiap iterasi.

Secara keseluruhan proses Backward Propagation dapat digambarkan sebagai berikut:

Output Prediksi --> Hitung Loss --> Hitung Error --> Hitung Gradien --> Update Bobot --> Training Berulang --> Model Optimal

Proses ini dilakukan berulang pada setiap epoch hingga model menghasilkan nilai loss yang minimum dan accuracy yang optimal.

B. Hasil Pengujian

Pada tahap ini dilakukan serangkaian eksperimen untuk memperoleh konfigurasi Artificial Neural Network (ANN) terbaik. Setiap eksperimen dievaluasi menggunakan metrik training loss, training accuracy, testing loss, dan testing accuracy. Selain itu dilakukan visualisasi menggunakan grafik loss, grafik accuracy, dan confusion matrix untuk melihat performa model secara lebih detail.

i. Pengaruh Depth Dan Width

Eksperimen pertama dilakukan untuk mengetahui pengaruh jumlah hidden layer (depth) dan jumlah neuron (width) terhadap performa model.

A. Eksperimen Hidden Layer

Hasil pengujian:

Hidden	Train Loss	Train Acc (%)	Test Loss	Test Acc (%)
1	0.774	85.57	0.7972	85.31
2	0.6545	86.62	0.5796	87.62
3	0.8354	85.52	0.8051	85.42

Analisis:

Peningkatan jumlah hidden layer dari 1 menjadi 2 meningkatkan kemampuan model dalam mempelajari pola data sehingga accuracy meningkat. Namun ketika jumlah hidden layer ditambah menjadi 3, performa justru menurun. Hal ini menunjukkan bahwa penambahan kompleksitas model tidak selalu menghasilkan performa yang lebih baik.

Konfigurasi terbaik: Hidden Layer = 2

B. Eksperimen Jumlah Neuron

Hasil pengujian:

Neuron	Train Loss	Train Acc (%)	Test Loss	Test Acc (%)
16	0.6945	76.73	0.6961	75.98
32	0.602	82.5	0.568	83.4
64	0.6877	82.83	0.684	82.56
128	0.7536	86.55	0.6857	86.96

Analisis:

Semakin banyak neuron memberikan kapasitas representasi yang lebih besar sehingga model mampu mempelajari pola yang lebih kompleks. Pada penelitian ini konfigurasi terbaik diperoleh pada 128 neuron.

Konfigurasi terbaik: Neuron = 128

ii. Pengaruh Fungsi Aktivasi

Eksperimen dilakukan menggunakan Sigmoid, ReLU, dan Tanh.

Hasil:

Aktivasi	Train Loss	Train Acc (%)	Test Loss	Test Acc (%)
sigmoid	0.7443	84.4	0.7475	84.58
relu	0.4175	89.53	0.4226	89.09
tanh	0.3762	89.82	0.371	90.08

Analisis:

Fungsi aktivasi Tanh menghasilkan performa terbaik karena distribusi output yang lebih seimbang dibanding Sigmoid dan ReLU.

Konfigurasi terbaik: Aktivasi = Tanh

iii. Pengaruh Learning Rate

Hasil:

LR	Train Loss	Train Acc (%)	Test Loss	Test Acc (%)
0.1	0.4067	88.3	0.4039	88.28
0.01	0.8781	72.11	0.8828	71.58
0.001	1.7494	42.44	1.7673	40.32

Analisis:

Learning rate sebesar 0,1 memberikan konvergensi paling cepat dan menghasilkan accuracy tertinggi.

Konfigurasi terbaik: Learning Rate = 0,1

iv. Pengaruh Inisialisasi Bobot

Hasil:

Init	Train Loss	Train Acc (%)	Test Loss	Test Acc (%)
zero	1.8441	26.41	1.8557	24.64
uniform	0.3783	89.39	0.3739	89.83
normal	0.7485	84.43	0.7818	83.77
xavier	0.4202	89.61	0.418	89.97
he	0.3495	89.71	0.35	89.35

Analisis:

Xavier menghasilkan distribusi bobot yang lebih stabil sehingga menghasilkan performa terbaik.

Konfigurasi terbaik: Inisialisasi = Xavier

v. Pengaruh Regularisasi

Pada penelitian ini tidak diterapkan metode regularisasi secara eksplisit seperti L1 Regularization, L2 Regularization, Dropout, maupun Early Stopping karena hasil evaluasi model menunjukkan bahwa performa training dan testing memiliki selisih yang relatif kecil sehingga tidak ditemukan indikasi overfitting yang signifikan.

Dengan demikian fokus eksperimen pada penelitian ini diarahkan pada optimisasi arsitektur dan hyperparameter seperti jumlah hidden layer, jumlah neuron, fungsi aktivasi, fungsi loss, learning rate, metode optimisasi, ukuran batch, dan jumlah epoch.

vi. Fungsi Loss

Eksperimen dilakukan untuk membandingkan dua fungsi loss yaitu Binary Cross Entropy (BCE) dan Multiclass Cross Entropy (MCE).

Hasil pengujian:

Analisis:

Loss	Train Loss	Train Acc (%)	Test Loss	Test Acc (%)
bce	0.0996	89.47	0.0994	89.42
mce	0.3938	89.04	0.3892	89.06

Secara numerik Binary Cross Entropy menghasilkan accuracy sedikit lebih tinggi dibandingkan Multiclass Cross Entropy. Namun pada penelitian ini Multiclass Cross Entropy tetap dipilih karena lebih sesuai secara teoritis untuk permasalahan klasifikasi multiclass yang menggunakan output Softmax.

Pada klasifikasi multiclass setiap data hanya memiliki satu kelas benar sehingga Multiclass Cross Entropy lebih merepresentasikan distribusi probabilitas antar kelas.

Konfigurasi terbaik: Loss Function = Multiclass Cross Entropy

vii. Pengaruh Ukuran Batch (Batch Size)

Eksperimen dilakukan menggunakan batch size 16, 32, dan 64.

Hasil pengujian:

Batch	Train Loss	Train Acc (%)	Test Loss	Test Acc (%)
16	0.1837	93.18	0.1944	93.06
32	0.1866	93.04	0.1922	93.35
64	0.1929	92.85	0.1968	93.13

Analisis:

Ketiga ukuran batch menghasilkan performa yang relatif mirip. Batch size 32 dipilih karena memberikan accuracy yang sama dengan batch size 16 namun lebih efisien secara komputasi.

Konfigurasi terbaik: Batch Size = 32

viii. Pengaruh Jumlah Epoch

Eksperimen dilakukan menggunakan 50, 100, dan 200 epoch.

Hasil pengujian:

Epoch	Train Loss	Train Acc (%)	Test Loss	Test Acc (%)
50	0.1947	92.81	0.197	92.91
100	0.1883	92.92	0.1937	93.24
200	0.185	93.12	0.1944	93.24

Analisis:

Peningkatan epoch dari 50 ke 100 meningkatkan performa model. Namun saat epoch ditingkatkan menjadi 200 tidak terjadi peningkatan accuracy testing meskipun training accuracy masih meningkat.

Hal ini menunjukkan munculnya indikasi overfitting ringan sehingga dipilih epoch 100.

Konfigurasi terbaik: Epoch = 100

ix. Metode Optimisasi

Eksperimen dilakukan menggunakan Full Batch Gradient Descent dan Mini Batch Gradient Descent.

Hasil pengujian :

Optimisasi	Train Loss	Train Acc (%)	Test Loss	Test Acc (%)
full_batch	0.3723	90.14	0.3719	89.83
mini_batch	0.187	93.02	0.1946	93.39

Mini Batch Gradient Descent menghasilkan performa lebih baik dibandingkan Full Batch. Loss menjadi lebih kecil dan accuracy meningkat baik pada data training maupun testing.

Perbedaan train dan test yang kecil menunjukkan bahwa model memiliki kemampuan generalisasi yang baik.

Konfigurasi terbaik: Mini Batch Gradient Descent

D. Perbandingan Dengan Library Keras

Sebagai pembandingan terhadap implementasi Artificial Neural Network (ANN) from scratch, dilakukan implementasi menggunakan library Keras. Tujuan perbandingan ini adalah untuk mengetahui perbedaan performa model, pengaruh hyperparameter, serta melihat apakah implementasi manual mampu menghasilkan performa yang kompetitif terhadap framework deep learning.

Pada implementasi Keras dilakukan eksperimen terhadap parameter yang sama seperti implementasi from scratch, yaitu :

a. jumlah hidden layer

```
...  
  
Hidden Layer = 1  
Accuracy: 0.9247  
  
Hidden Layer = 2  
Accuracy: 0.9269  
  
Hidden Layer = 3  
Accuracy: 0.931
```

b. jumlah neuron

```
Neuron = 16  
Accuracy: 0.9218  
  
Neuron = 32  
Accuracy: 0.9277  
  
Neuron = 64  
Accuracy: 0.9291  
  
Neuron = 128  
Accuracy: 0.931
```

c. learning rate

```
...  
  
Learning Rate = 0.1  
Accuracy: 0.9163  
  
Learning Rate = 0.01  
Accuracy: 0.9306  
  
Learning Rate = 0.001  
Accuracy: 0.9313
```

d. jumlah epoch

```
...  
  
Epoch = 50  
Accuracy: 0.9346  
  
Epoch = 100  
Accuracy: 0.9277  
  
Epoch = 200  
Accuracy: 0.9379
```

e. batch size

```
...  
  
Batch Size = 16  
Accuracy: 0.9328  
  
Batch Size = 32  
Accuracy: 0.9273  
  
Batch Size = 64  
Accuracy: 0.9221
```

f. inisialisasi bobot.

```
...  
  
Zero  
Accuracy: 0.2604  
  
Uniform  
Accuracy: 0.9291  
  
Normal  
Accuracy: 0.9328  
  
Xavier  
Accuracy: 0.9221  
  
He  
Accuracy: 0.928
```

Ringkasan konfigurasi terbaik pada implementasi Keras:

- Hidden Layer = 3 (Accuracy = 93,10%)
- Neuron = 128 (Accuracy = 93,10%)
- Learning Rate = 0,001 (Accuracy = 93,13%)

- Epoch = 200 (Accuracy = 93,79%)
- Batch Size = 16 (Accuracy = 93,28%)
- Inisialisasi Bobot = Normal (Accuracy = 93,28%)

Sedangkan konfigurasi terbaik pada implementasi ANN from scratch adalah:

- Hidden Layer = 2 (Accuracy = 87,62%)
- Neuron = 128 (Accuracy = 86,96%)
- Learning Rate = 0,1 (Accuracy = 88,28%)
- Epoch = 100 (Accuracy = 93,24%)
- Batch Size = 32 (Accuracy = 93,24%)
- Inisialisasi Bobot = Xavier (Accuracy = 89,97%)

Hasil menunjukkan bahwa implementasi menggunakan Keras menghasilkan accuracy sedikit lebih tinggi dibanding implementasi from scratch.

Perbedaan ini dapat terjadi karena Keras telah mengoptimalkan proses komputasi jaringan saraf secara internal, termasuk proses inisialisasi parameter, stabilitas numerik, optimisasi training, serta pengelolaan operasi matriks yang lebih efisien.

E. Kesimpulan Dan Saran

a. Kesimpulan

Berdasarkan seluruh eksperimen yang dilakukan diperoleh konfigurasi Artificial Neural Network terbaik sebagai berikut:

From Scratch	Library Keras
• Hidden Layer = 3 (Accuracy = 93,10%) • Neuron = 128 (Accuracy = 93,10%) • Learning Rate = 0,001 (Accuracy = 93,13%) • Epoch = 200 (Accuracy = 93,79%) • Batch Size = 16 (Accuracy = 93,28%) • Inisialisasi Bobot = Normal (Accuracy = 93,28%)	• Hidden Layer = 2 (Accuracy = 87,62%) • Neuron = 128 (Accuracy = 86,96%) • Learning Rate = 0,1 (Accuracy = 88,28%) • Epoch = 100 (Accuracy = 93,24%) • Batch Size = 32 (Accuracy = 93,24%) • Inisialisasi Bobot = Xavier (Accuracy = 89,97%)

Dari hasil seluruh eksperimen diatas, Keras memang menunjukkan performa lebih dibandingkan from Scratch, akan tetapi implementasi ANN from scratch tetap mampu menghasilkan performa yang sangat kompetitif dengan selisih yang relatif kecil.

Selain memperoleh accuracy yang mendekati Keras, implementasi manual memberikan pemahaman yang lebih mendalam terhadap proses kerja jaringan saraf mulai dari forward propagation, fungsi loss, backward propagation, hingga update parameter.

b. Saran

Dari semua hasil yang sudah didapat saran dari saya adalah jika tujuan utama membangun ANN adalah memperoleh performa dan efisiensi implementasi, maka penggunaan Keras lebih direkomendasikan. Namun jika tujuan utama adalah memahami konsep dasar Artificial Neural Network secara matematis dan implementatif, maka pendekatan from scratch memberikan nilai pembelajaran yang lebih tinggi.

F. Referensi

“I Built a Neural Network from Scratch” by Green Code
https://youtu.be/cAkMcPfY_Ns?si=oM8VxWYAVLm9zHoQ

“Building a neural network FROM SCRATCH (no Tensorflow/Pytorch, just numpy & math)” by Samson Zhang
<https://youtu.be/w8yWXqWQYmU?si=ov1NpSUBVah8AnZL>

Playlist On YouTube (1-9) “Neural Networks from Scratch in Python” by Sentdex
https://youtube.com/playlist?list=PLQVvva0QuDcjD5BAw2DxE6OF2tius3V3&si=3BF17e_C7vHEYwAg

“P2.3.6 Build Your First ANN from Scratch — Deep Learning” by MultiMagix
https://youtu.be/RwkNDgMAv_k?si=2ecwLCutW44aIbKT

Customer churn prediction using ANN | Deep Learning Tutorial 18 (Tensorflow2.0, Keras & Python) by codebasics
<https://youtu.be/MSBY28IJ47U?si=4RpM5OHca8ZXaBi>

Backpropagation Using Python- RB Fajriya Hakim
<https://medium.com/@986110101/backpropagation-using-python-04781245693b>

Dasar Python, Ahmad Z. Ihsan <https://az-ihsan.github.io/basic-python-udn/>

Jaringan saraf buatan- Ashish Wakde
<https://id.linkedin.com/pulse/artificial-neural-network-ann-bank-customer-data-analysis-wakde-c9akf?tl=id>

“Introduction to Neural Network Operations” by 3Blue1Brown
<https://codesignal.com/learn/courses/enigmatic-autoencoders-for-dimensionality-reduction-1/lessons/neural-network-forward-propagation-1>

G. LAMPIRAN

The first screenshot shows the implementation of forward propagation for a Fully Connected Layer (FCL). The code defines a function `forward_fcl(X, W, b)` that performs a linear operation $Z = (X @ W + b)$ and returns `Z`. It also initializes parameters `W` and `b` using `uniform_init` and `zero_init` respectively. The second screenshot shows the implementation of the output layer. It defines `output_size = 7`, initializes `W2` and `b2`, and performs the forward pass `Z2 = forward_fcl(A, W2, b2)`. The output `A2` is then calculated using the softmax activation function. The final output shape is `(10888, 7)`.

```
# Implementasi Forward Propagation untuk FCL
def forward_fcl(X, W, b):
    # Operasi linear
    Z = (X @ W + b)
    return Z

# Inisialisasi Parameter (IMPLEMENTASI AWAL)
W = uniform_init((16,16))
b = zero_init((1,16))

# Fungsi Aktivasi Hidden
def sigmoid(Z):
    return (1 / (1 + np.exp(-Z)))

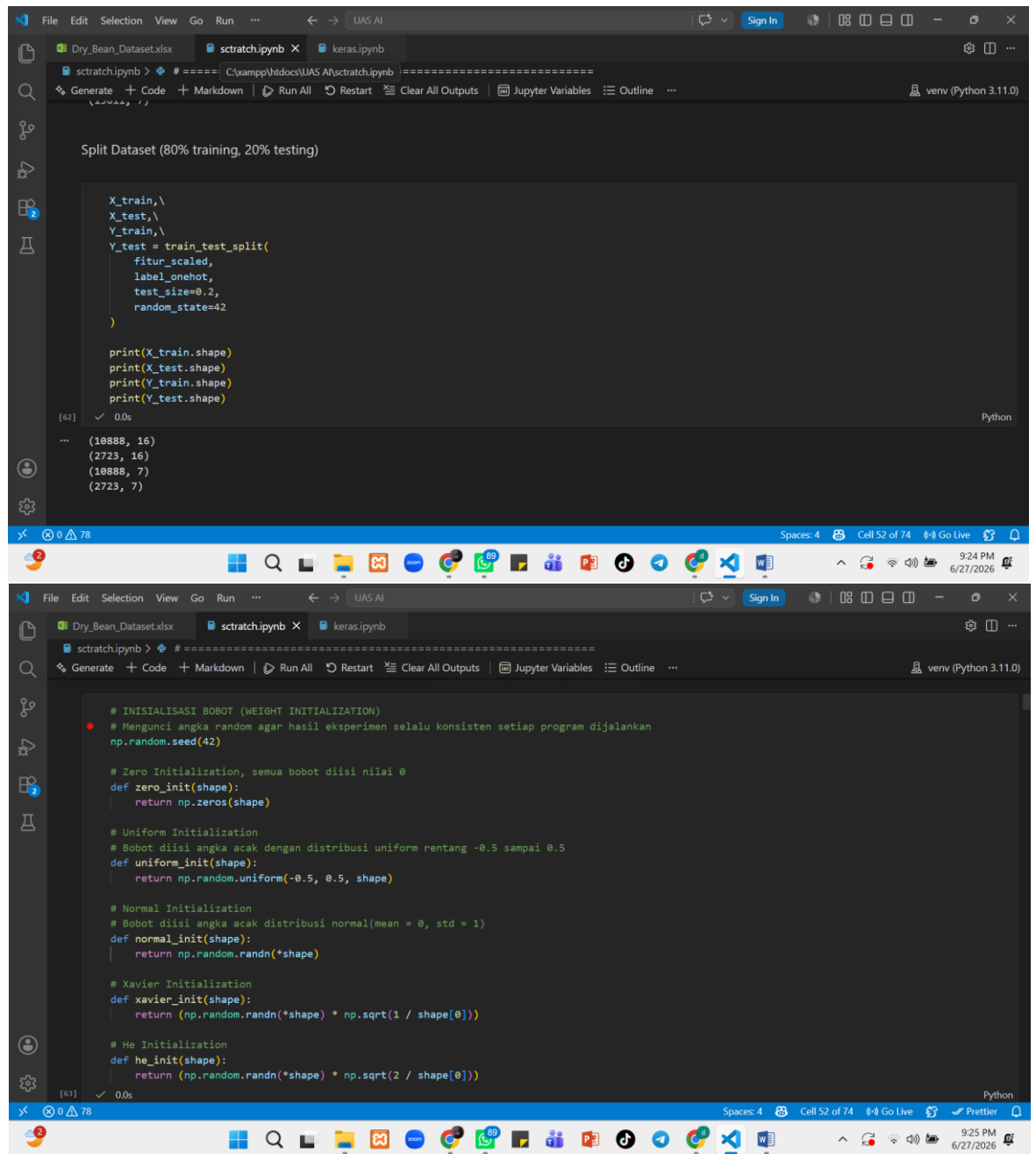
def relu(Z):
    return np.maximum(0,Z)
```

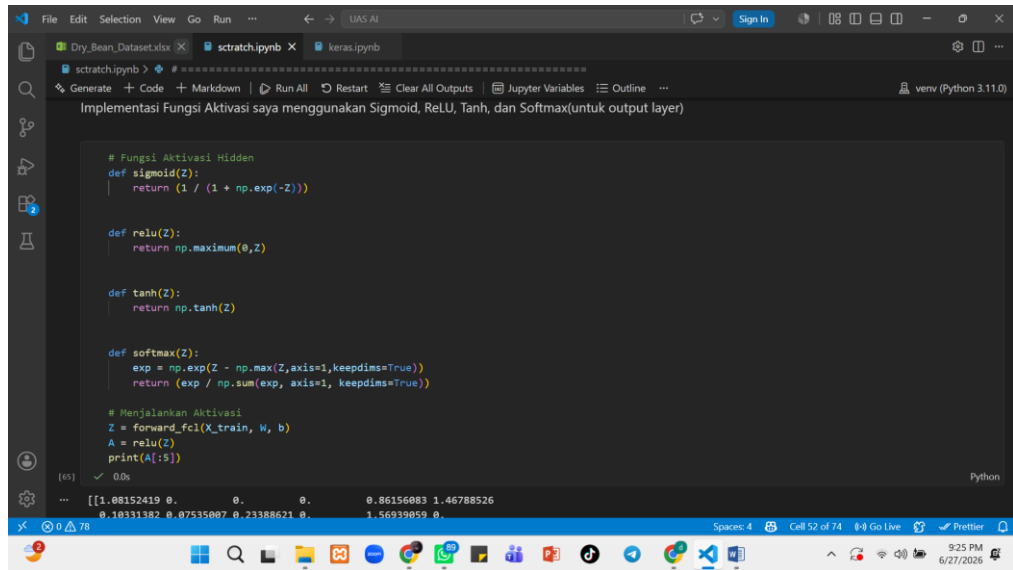
```
# Implementasi FCL Output Layer
output_size = 7
# Membuat bobot output
W2 = he_init((A.shape[1], output_size))

# Membuat bias output
b2 = np.zeros((1, output_size))
# Forward FCL Output
Z2 = forward_fcl(A, W2, b2)

# Aktivasi Output (Softmax)
A2 = softmax(Z2)

# Cek hasil
print("Shape Output:", A2.shape)
print()
print(A2[:5])
```





The image shows a Jupyter Notebook titled "sctrach.ipynb" in a VS Code editor. The notebook contains Python code for implementing Sigmoid, ReLU, Tanh, and Softmax activation functions. The code is as follows:

```
# Fungsi Aktivasi Hidden
def sigmoid(Z):
    return 1 / (1 + np.exp(-Z))

def relu(Z):
    return np.maximum(0,Z)

def tanh(Z):
    return np.tanh(Z)

def softmax(Z):
    exp = np.exp(Z - np.max(Z,axis=1,keepdims=True))
    return (exp / np.sum(exp,axis=1,keepdims=True))

# Menjalankan Aktivasi
Z = forward_fc1(X_train, W, b)
A = relu(Z)
print(A[:5])
```

The output of the code is displayed in the console:

```
[[1.08152419 0. 0. 0. 0.86156083 1.46788526
 0.10331182 0.07535087 0.23388621 0. 1.56939059 0.]
```

The bottom status bar of the VS Code window shows "Spaces: 4", "Cell 52 of 74", and the time "9:25 PM 6/27/2026".